DIGITAL ACADEMIC DOCUMENT WORKFLOW SYSTEM

A ROLE-BASED PLATFORM FOR ONLINE ACADEMIC REQUESTS, APPROVAL ROUTING AND VERIFIED DOCUMENTS

A PROJECT REPORT

Submitted by

GOKUL R — 310624148027

in partial fulfillment of the requirements for the degree of

BACHELOR OF ENGINEERING

Computer Science and Engineering

(Artificial Intelligence and Machine Learning)

2311CSE503R — FULL STACK DEVELOPMENT



EASWARI ENGINEERING COLLEGE

(AN AUTONOMOUS INSTITUTION) — AFFILIATED TO ANNA UNIVERSITY

RAMAPURAM, CHENNAI

OCTOBER 2026

EASWARI ENGINEERING COLLEGE

BONAFIDE CERTIFICATE

This is to certify that the project report entitled **"DIGITAL ACADEMIC DOCUMENT WORKFLOW SYSTEM: A ROLE-BASED PLATFORM FOR ONLINE ACADEMIC REQUESTS, APPROVAL ROUTING AND VERIFIED DOCUMENTS"** is submitted by **GOKUL R (310624148027)**, in partial fulfillment of the requirements for the award of the degree of Bachelor of Engineering in Computer Science and Engineering (Artificial Intelligence and Machine Learning).

This report describes the project implementation and a clearly identified synthetic dataset prepared for demonstration. Faculty review and signatures are pending.

HEAD OF THE DEPARTMENT	PROJECT SUPERVISOR
Name: _____	Name: _____
Signature: _____	Signature: _____

EASWARI ENGINEERING COLLEGE

CERTIFICATE OF VIVA-VOCE EXAMINATION

This is to certify that the Viva-Voce Examination of the project work entitled **"Digital Academic Document Workflow System"**, submitted by **GOKUL R** (Register No. **310624148027**), in partial fulfillment of the requirements for the award of the degree of Bachelor of Engineering in Computer Science and Engineering (Artificial Intelligence and Machine Learning), was conducted on _____ at Easwari Engineering College, Ramapuram, Chennai.

INTERNAL EXAMINER	EXTERNAL EXAMINER
Name: _____	Name: _____
Designation: _____	Designation: _____
Signature: _____	Signature: _____

Date: _____

ABSTRACT

Colleges process bonafide certificates, transcripts, internship no-objection certificates and recommendation letters through paper forms, email and in-person signatures. Students may have difficulty knowing where a request is waiting, while staff must manually forward requests and preserve an audit trail. The Digital Academic Document Workflow System (DocFlow) is a web application that digitizes this process. A student selects an active document type, completes its configurable form and submits a request. The system routes the request through approval stages defined by an administrator, such as Class Advisor, Head of Department and Academic Office. Each approval, rejection or revision request is recorded with the acting user, stage, comment and timestamp. Role-specific pages present students with their requests, approvers with pending work, and administrators with user, document-type and request-management views. The application follows a MERN-style architecture. A React and Vite client communicates with an Express REST API. MongoDB documents are managed through Mongoose models. Authentication uses JSON Web Tokens, passwords are hashed with bcrypt, and server middleware checks roles before protected operations. On completion, the application generates a PDF containing a unique document identifier and QR code; a public verification view checks the issued document against stored records. Persistent in-app notifications and configurable email integration provide status updates. The current demonstration environment includes four active request types and a synthetic dataset: 40 demo accounts across four departments, 120 sample requests dated from August 2025 through October 2026, 68 generated sample PDFs, and 49 sample notifications. These records demonstrate application workflows and are not evidence of real college usage. The system provides a practical foundation for replacing manual routing with traceable, configurable digital processing; production deployment would require institutional account data, faculty approval, mail configuration, operational security review and backup procedures.

Keywords: academic workflow, document request, role-based access control, approval chain, QR verification, MERN stack

ACKNOWLEDGEMENT

I express my sincere gratitude to the management and Principal of Easwari Engineering College for providing the academic environment and resources needed to undertake this project.

I thank the Head of the Department and the faculty of Computer Science and Engineering (Artificial Intelligence and Machine Learning) for their encouragement and the opportunity to develop a full-stack academic workflow application.

I am grateful to the project supervisor for guidance, review and suggestions during planning and implementation. The supervisor name and approval are intentionally left for the department to complete in the certificate pages.

I acknowledge the maintainers of the open-source technologies used in the implementation, including React, Vite, Node.js, Express, MongoDB, Mongoose, bcrypt, PDFKit and QRCode.

Finally, I thank my family and friends for their support throughout the project.

TABLE OF CONTENTS

CHAPTER 1INTRODUCTION 10

1.1 Overview 10

1.2 Problem Definition 10

1.3 Project Objectives 10

1.4 Scope of the Project 10

1.5 Intended Users 11

1.6 Organization of the Report 11

CHAPTER 2RELATED WORK AND PROJECT CONTEXT 12

2.1 Manual Academic Processing 12

2.2 Digitized Forms and Tracking 12

2.3 Digital Issuance and Verification 12

2.4 Comparison of Process Characteristics 12

2.5 Design Principles 12

2.6 Limitations of the Prototype 12

CHAPTER 3SYSTEM ANALYSIS 13

3.1 Functional Requirements 13

3.2 Non-Functional Requirements 13

3.3 Hardware and Software Requirements 13

3.4 Feasibility 13

3.5 Assumptions and Constraints 13

CHAPTER 4SYSTEM DESIGN 14

4.1 Architectural Design 14

4.2 Request Workflow 15

4.3 Role and Permission Model 15

4.4 Data Model 16

4.5 Request Status Model 16

CHAPTER 5IMPLEMENTATION AND DEMONSTRATION 17

5.1 Frontend Implementation 17

5.2 Backend Implementation 17

5.3 Authentication and Access Control 17

5.4 Configurable Document Types 17

5.5 Workflow Routing and Notifications 17

5.6 PDF Generation and QR Verification 17

5.7 API Surface 17

5.8 Source Layout 18

5.9 Verification Approach 19
5.10 Observed Local Demonstration 19
5.11 Demonstration Dataset Integrity 19
5.12 Evaluation 19
5.13 Known Limitations 20

CHAPTER 6CONCLUSION AND FUTURE WORK 21
6.1 Conclusion 21
6.2 Future Work 21

LIST OF ABBREVIATIONS 22

REFERENCES 23

APPENDIX A — LOCAL STARTUP AND REVIEW 24
A.1 Start the Local Application 24
A.2 Administrator Review Sequence 24
A.3 Repository Structure 24

APPENDIX B — SAMPLE WORKFLOW RECORD 25

LIST OF FIGURES

Figure 4.1: Three-tier DocFlow application architecture

Figure 4.2: Configurable request and issuance sequence

Figure 4.3: Core entities and relationships

LIST OF TABLES

Table 2.1: Manual, form-only and DocFlow process comparison

Table 3.1: Functional requirements

Table 3.2: User roles and permissions

Table 3.3: System requirements

Table 4.1: Core data entities

Table 4.2: Request status model

Table 5.1: API route groups

Table 5.2: Verification scenarios

Table 5.3: Demonstration dataset summary

CHAPTER 1
INTRODUCTION

1.1 Overview

Academic document requests often pass through multiple people before a certificate or record can be issued. A student may submit a paper form, carry it to a class advisor, wait for a department signature and then visit an administrative office. At each hand-off, students and staff need to know which version is current and who is responsible for the next action. A digital workflow can make these steps visible and record each decision in one place. DocFlow is a role-based web application for submitting, reviewing and issuing academic documents. Administrators configure document types, form fields and approval chains. Students submit requests; designated approvers act on them; the system preserves a time-stamped history. Requests that complete their configured stages can produce a PDF with a unique identifier and a QR-based verification link.

1.2 Problem Definition

Paper-based academic document processing creates avoidable delays and uncertainty. Requests can be misplaced, approval responsibility may be unclear, and students often need to ask staff for status updates. A paper trail also makes it difficult for administrators to review turnaround, pending queues or past decisions. The problem addressed is: *How can a college provide a configurable, auditable online process for academic-document requests that routes each request to the correct approvers and gives students a verifiable digital outcome?*

1.3 Project Objectives

The system is designed to:

- Provide authenticated access for students, Class Advisors, HODs, Deans, Academic Office Staff and Super Administrators.
- Let administrators create academic accounts and manage user status.
- Represent each document type with configurable form fields and an ordered approval chain.
- Route submitted requests to an active approver and preserve stage-level comments and timestamps.
- Allow an approver to approve, reject or request a revision.
- Notify users in-app and support email notifications when configured.
- Generate a PDF for completed requests and provide a public QR/document verification flow.
- Give students, approvers and administrators role-specific views of work and activity.

1.4 Scope of the Project

The application covers account management, authentication, document-type configuration, student request creation, approval queues, request histories, in-app notifications, PDF generation and QR verification. The bundled document types are Bonafide Certificate, Transcript Request, Internship No Objection Certificate and Recommendation Letter. Administrators can adapt fields and approval stages to local procedures. The project is a college workflow prototype. It does not connect to a student information system, e-signature provider, payment platform or official college directory. Email delivery depends on SMTP configuration. Demonstration records are synthetic and do not represent actual students or completed college transactions.

1.5 Intended Users

Students initiate and monitor requests. Class Advisors and HODs review department-specific work. Deans and Academic Office Staff may participate where a document type includes those stages. Super Administrators configure the application, manage accounts and review overall activity. The backend enforces permissions as well as the user interface.

1.6 Organization of the Report

Chapter 2 presents workflow context and solution patterns. Chapter 3 records requirements, constraints and feasibility. Chapter 4 describes architecture, workflow and data design. Chapter 5 describes implementation and demonstration. Chapter 6 concludes the report and presents future extensions. Appendices provide local operating guidance, abbreviations and source references.

CHAPTER 2
RELATED WORK AND PROJECT CONTEXT

2.1 Manual Academic Processing

A conventional process uses forms, signatures, office visits and email. It is familiar and can work for small volumes, but status is distributed across paper files and individual inboxes. Students may not know whether a request is awaiting an advisor, a department head or an office clerk. If a form is incomplete, the applicant may only learn this after a later review.

2.2 Digitized Forms and Tracking

A basic online form simplifies collection but does not, by itself, coordinate approvals. A useful academic workflow needs a record for each request, a configurable sequence of approvers, a queue for each role, a clear decision history and notifications when responsibility changes. DocFlow combines request entry with routing and audit functions.

2.3 Digital Issuance and Verification

An issued PDF is easier to distribute than a paper copy, but a file alone does not prove that it came from the college. DocFlow associates the generated document with a unique identifier and a QR code that opens a verification route. The verification view queries the stored generated-document record. This is an application-level check; it is not a cryptographic signature or an external registry integration.

2.4 Comparison of Process Characteristics

The comparison describes project design goals, not a benchmark of named commercial products.

CAPABILITY	MANUAL PROCESS	FORM-ONLY PORTAL	DOCFLOW DESIGN
Request intake	Paper or email	Online form	Dynamic form per document type
Routing	Physical hand-off	Often separate/manual	Configured approval stages
Status visibility	Enquiry required	Submission receipt	Role dashboards and request timeline
Decision record	Paper notes	May be limited	Actor, stage, action, comment and timestamp
Document output	Printed copy	Manual attachment	Generated PDF with ID and QR verification

2.5 Design Principles

The project follows four practical principles: keep the request history attached to the request; store approval stages as configuration rather than hard-coded screen logic; check role permissions at the API boundary; and generate the issued document from the final workflow record. This makes the process easier to inspect and extend than a collection of disconnected forms.

2.6 Limitations of the Prototype

The demonstration does not use live institutional identities or an official student roster. It relies on user-entered department strings, so naming must be standardized before deployment. Workflow availability depends on an active user for each required role and department. The QR code validates a record stored by this application; it does not independently verify an individual’s identity.

CHAPTER 3
SYSTEM ANALYSIS

3.1 Functional Requirements

The requirements below describe intended application behaviour, not a claim that every external integration is configured in the local demonstration.

ID	REQUIREMENT
FR-01	Authenticate users and expose role-appropriate application routes.
FR-02	Create, search, update and deactivate academic user accounts.
FR-03	Register students with department, year, section and roll number.
FR-04	Create document types with custom fields and ordered approval stages.
FR-05	Submit a request and assign it to the next eligible active approver.
FR-06	Record approvals, rejections and revision requests with comments and timestamps.
FR-07	Show request queues, request details, status and history.
FR-08	Create persistent in-app notifications for workflow events.
FR-09	Generate a PDF and store an associated unique verification record.
FR-10	Show dashboard summaries for users, requests, pending work and active document types.

3.2 Non-Functional Requirements

The application separates responsibilities between client, REST API and persistence; validates inputs and reports errors; restricts protected API operations by role; and presents usable pages on desktop and mobile browsers. Credentials must not be stored as plain text. Generated files and database records should remain available across local restarts. Institutional deployment also requires backup, access review, monitoring and retention procedures.

3.3 Hardware and Software Requirements

The Windows launcher uses Node.js for the client and API and a persistent local MongoDB-compatible database when an external MongoDB server is not available. A modern browser is used to access the React client. An institutional deployment should use supported server hardware, managed MongoDB or a hardened database host, TLS termination, backups and operational monitoring. Production sizing depends on volumes and retention, which have not been measured.

3.4 Feasibility

Technical: the MERN stack supports user, form, workflow and document operations. **Operational:** role-specific queues correspond to existing college responsibilities and can reduce repeated status enquiries. **Economic:** the prototype uses open-source libraries and local hosting; production hosting and email costs depend on infrastructure. **Schedule:** client, API, models and services allow incremental development.

3.5 Assumptions and Constraints

Users are assigned a department and role by an administrator. Department names must match across students, advisors and HODs for routing. Each approval chain must have an active eligible approver before requests can progress. Email depends on SMTP configuration. The application operates as a local demonstration at 127.0.0.1 unless deployed separately. This report describes the repository inspected on 21 October 2026.

CHAPTER 4
SYSTEM DESIGN

4.1 Architectural Design

The application separates the browser interface from the API and persistence services. The client requests data over REST endpoints. Express authenticates requests, checks authorization, invokes controllers and services, and reads or writes MongoDB documents through Mongoose. Document generation and QR creation run as server-side services.

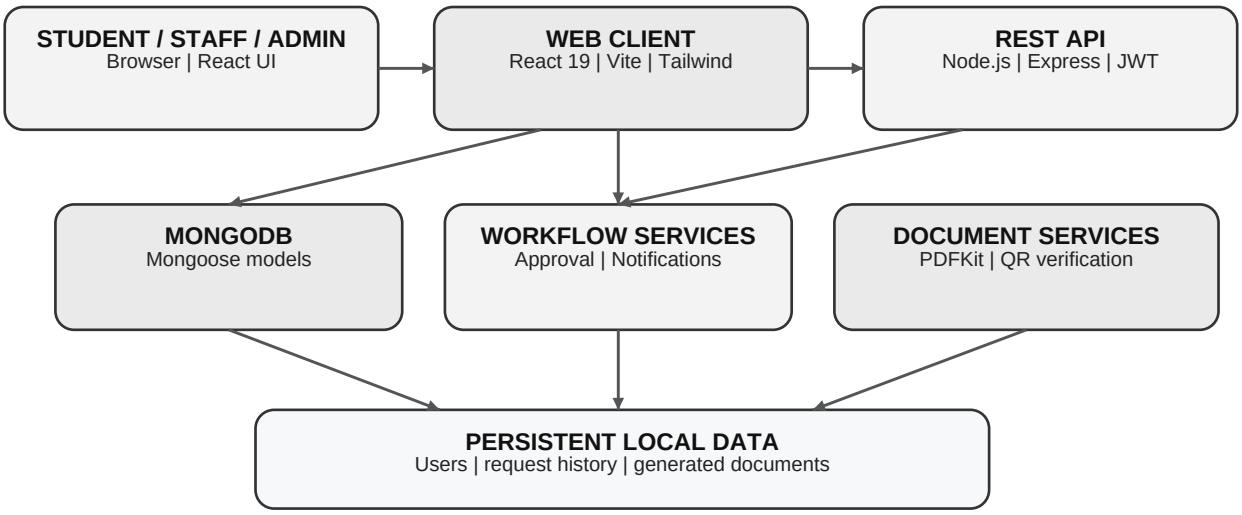


Figure 4.1: Three-tier DocFlow application architecture

4.2 Request Workflow

Document-type configuration determines the approval chain. A request stores its current stage and approver, together with a chronological history. The workflow service finds an active approver by role and, for department roles, by the student's department and class details. If an approver is missing, the application reports that the request cannot safely progress.

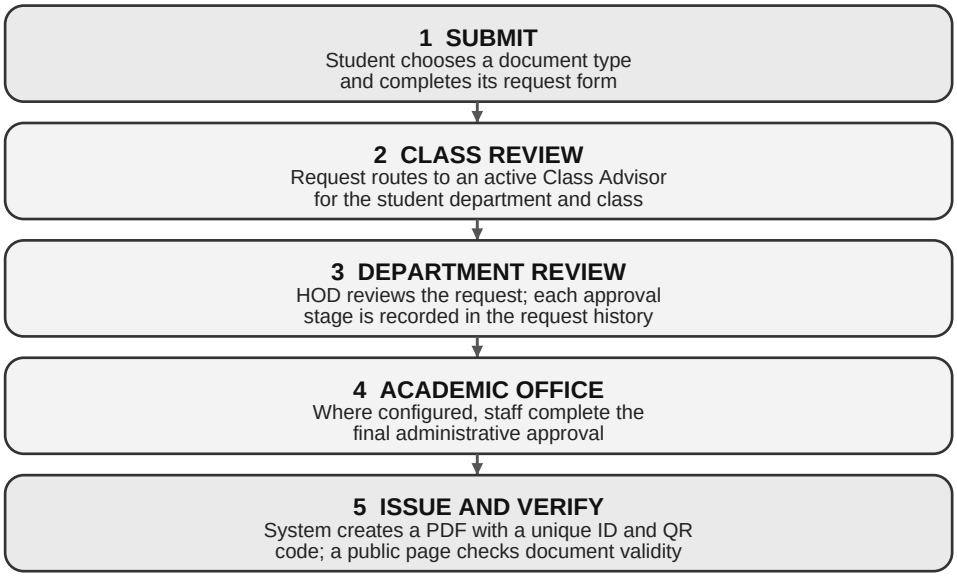


Figure 4.2: Configurable request and issuance sequence

4.3 Role and Permission Model

The backend protects administrator endpoints and applies role checks to request and approval routes. A role's appearance in a menu is not the only control; protected API routes also enforce authorization.

ROLE	PRIMARY RESPONSIBILITY
Student	Register, submit requests, view own requests and download issued documents.
Class Advisor	Review requests for the assigned department/class and approve, reject or request revision.
HOD	Review department requests at the configured department-head stage.
Dean	Participate when a document type's chain includes a Dean stage.
Admin Staff	Manage administrative queues, users and document configuration as permitted.
Super Admin	Configure the system and manage academic accounts, including privileged roles.

4.4 Data Model

MongoDB stores separate collections for users, document types, requests, generated documents and notifications. References connect a request to its student and document type, a generated document to its originating request, and a notification to its recipient and (optionally) a related request.

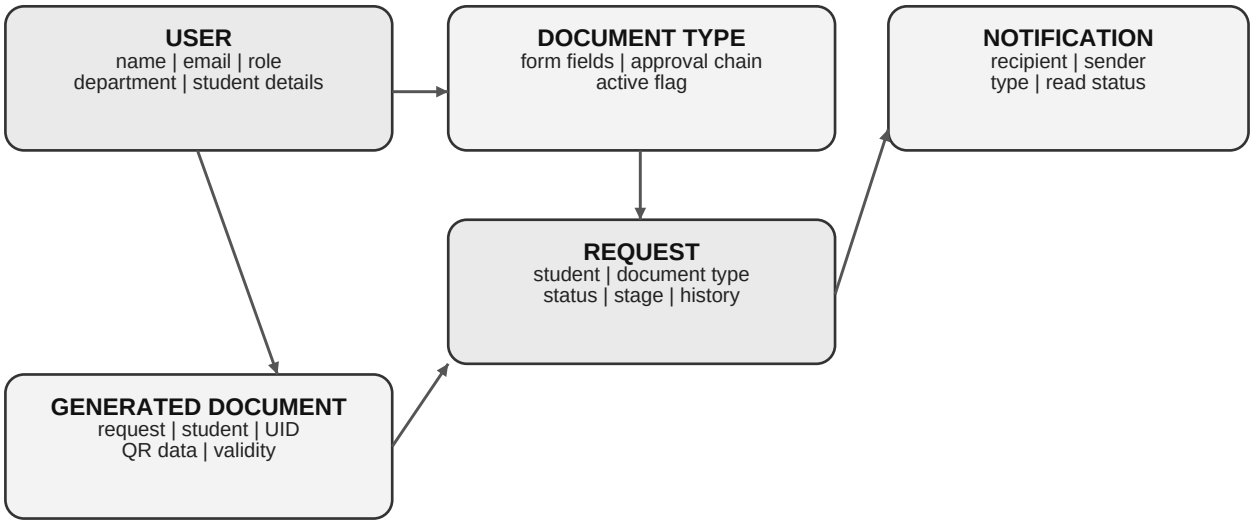


Figure 4.3: Core entities and relationships

ENTITY	IMPORTANT FIELDS / PURPOSE
User	Name, unique email, password hash, role, department and active flag; students also have year, section and roll number.
DocumentType	Name, category, description, dynamic custom fields, ordered approval chain and active flag.
Request	Student, document type, form data, status, current stage/approver, priority, document ID and action history.
GeneratedDocument	Request/student references, unique document UID, QR data, file path, issuer and validity.
Notification	Recipient, sender, event type, title/message, related request and read status.

4.5 Request Status Model

The schema supports draft, submitted, in_review, approved, rejected, revision_requested and completed. A request begins as submitted, moves into review as approvers act, and becomes completed when the final configured approval produces an issued PDF. Rejections stop routing; revision requests return a request for correction. The history preserves the action sequence after the current status changes.

CHAPTER 5
IMPLEMENTATION AND DEMONSTRATION

5.1 Frontend Implementation

The client is a React single-page application built with Vite and styled with Tailwind CSS. React Router exposes pages for login, student registration, password reset, dashboards, user administration, document types, request creation, pending approvals, request history, profile and public document verification. Axios modules centralize API communication. Toast notifications communicate operation results. The administrator dashboard summarizes users, requests, pending requests and active document types.

5.2 Backend Implementation

The API uses Express routes, controllers, Mongoose models, middleware and services. Helmet and CORS configure common HTTP protections and allowed local origins. The global error handler returns API errors. A health endpoint reports whether the database connection is ready. The Windows launcher copies application source to a non-synchronized runtime directory and keeps local database files and generated PDFs between restarts.

5.3 Authentication and Access Control

The authentication controller verifies credentials and issues a JWT. Passwords are stored as bcrypt hashes. The protect middleware validates a token and resolves the current user; authorize middleware restricts selected routes to permitted roles. Student requests are scoped to the student, while administration operations require an administrative role. Password-reset routes are present; production use should configure a secure delivery channel and review token expiry and reset policies.

5.4 Configurable Document Types

Document types declare a name, category, description, dynamic fields and approval chain. The client can render forms from those field definitions rather than using one fixed form for every request. The initial set contains Bonafide Certificate, Transcript Request, Internship No Objection Certificate and Recommendation Letter. Administrators can review or update configuration through Document Types pages.

5.5 Workflow Routing and Notifications

When a student submits a request, the workflow service resolves an active first-stage approver. Department-scoped stages match the student’s department; Class Advisor routing can also match year and section. Approvers act through approval endpoints. The application records stage, action, comment, performer and timestamp in request history, then routes to the next configured approver or ends the workflow. In-app notification records support workflow events; SMTP-backed email functions are included and require deployment configuration.

5.6 PDF Generation and QR Verification

For a completed approval chain, the server generates a PDF using PDFKit and creates QR content from the document UID. A GeneratedDocument record associates the UID, request, student, document type, file path, issuer and verification URL. The public verification page uses this record to display validity. This does not add a certificate-authority signature to the PDF.

5.7 API Surface

The implemented route groups are summarized below; exact payloads are defined by controllers and schemas.

ROUTE GROUP	EXAMPLE OPERATIONS
/api/auth	Login, student registration, current-user profile, forgot/reset password.
/api/admin	User management, document-type management and dashboard statistics.
/api/requests	Available types, request creation, own requests, details and admin request list.
/api/approvals	Pending approval queues and approve/reject/revision actions.
/api/notifications	List notifications, unread count, mark read and delete.
/api/documents	Download issued document and verify a UID.
/api/health	Database readiness check.

5.8 Source Layout

The repository uses a client/server split. The server contains config, controllers, middleware, models, routes and services. The client contains API modules, authentication context, reusable layout/components and pages grouped by role or function. The Windows launcher is start-app.ps1; the README documents startup and local runtime storage.

Key implementation files: server/models/User.js, DocumentType.js, Request.js, GeneratedDocument.js and Notification.js; server/services/workflow.service.js and pdf.service.js; server/controllers/admin.controller.js and request.controller.js; client/src/pages/admin, dashboard, requests and verify.

5.9 Verification Approach

Verification should cover authentication, role restrictions, dynamic form validation, department-specific routing, each approval outcome, request-history persistence, PDF creation, QR verification and notification read state. The local demonstration was checked against the live dashboard and account-management page after sample records were inserted. This report does not claim a formal automated test suite or production security audit.

AREA	VERIFICATION SCENARIO
Authentication	Valid sign-in returns the authenticated role; invalid credentials are rejected.
Authorization	Student credentials cannot invoke admin-only user or document-type operations.
Routing	A student request resolves to an active advisor and then follows its configured chain.
History	Each decision stores performer, action, stage, comment and chronological timestamp.
Completion	Final approval creates a generated-document record and downloadable PDF.
Verification	A valid UID resolves to a valid record; unknown IDs are not reported as issued.
Failure handling	Missing approver, invalid fields and database outage return a clear API error.

5.10 Observed Local Demonstration

At the time of report preparation, the local dashboard displayed 41 users, 120 requests, 32 pending requests and 4 active document types. The user-management page showed synthetic accounts as active. The local API health endpoint reported the database as ready. These checks confirm that seeded records are visible in the local application; they do not represent real institutional activity.

SAMPLE DATA ITEM	COUNT / RANGE
Demo accounts added	40: 25 students (15 girl-coded, 10 boy-coded names), 4 Class Advisors, 4 HODs, 7 Academic Office Staff
Pre-existing administrator	1 Super Admin
Departments represented	4
Demo requests	120; dated 7 August 2025 through 21 October 2026
Completed / generated sample PDFs	68
Rejected / revision requested	12 / 8
Submitted / in review	16 / 16
Demo notifications	49

5.11 Demonstration Dataset Integrity

Student names are randomly generated fictional names. Accounts use college-domain-style email addresses and distinct 14-digit IDs modeled on the supplied roll-number pattern. Names and identifiers are fictional; demo request comments and form fields mark the workflow records as sample data. The shared temporary demo password is for local demonstration only and is not printed in this report. These records were generated to show screens and workflows; they must not be described as real student accounts or institutional transactions.

5.12 Evaluation

The prototype demonstrates user roles, configurable request types, stage-based approval, persistent history, PDF output and QR verification. Admin summaries and request lists provide a starting point for monitoring activity. No measured turnaround-time or usability study is reported because the dataset is synthetic. Future evaluation should use approved test participants and a defined task set, with anonymized measures.

5.13 Known Limitations

Department names are free text and can diverge if staff enter inconsistent values. Email delivery depends on SMTP configuration. The local application uses a persistent embedded MongoDB-compatible runtime and is not configured as a production service. The QR code points to the configured client URL, so deployment must use the institution’s HTTPS domain. Operational policies are needed for retention, backup, account lifecycle and document corrections.

CHAPTER 6
CONCLUSION AND FUTURE WORK

6.1 Conclusion

The Digital Academic Document Workflow System provides a unified web application for academic document requests. It combines configurable forms, role-based accounts, sequential approvals, an auditable request timeline, in-app notifications and PDF issuance with QR verification. Its modular MERN implementation gives the college a practical prototype for exploring a paperless approval process. The local demonstration contains a deliberately synthetic year-plus dataset so administrators can inspect populated dashboards and workflow histories. The sample is clearly identified and does not imply that real institutional records have been processed. Before operational use, the college must approve workflows, replace demo accounts with verified identities, configure production hosting and email, and establish governance for records and generated documents.

6.2 Future Work

The following extensions would strengthen the system for institutional deployment:

- Use a controlled department/class directory and validate approver assignments against it.
- Integrate with the college identity provider or student information system after authorization and privacy review.
- Add deadline rules, escalation reminders and configurable service-level metrics.
- Add signed or tamper-evident documents, key rotation and an institutional verification domain.
- Version workflows so changes do not alter the meaning of older requests.
- Add automated backups, restore drills, audit export and retention controls.
- Run accessibility review and usability sessions with students and staff.
- Build automated tests for authorization, routing, request history, document generation and public verification.
- Replace synthetic demo data with approved, minimal test data and document a cleanup procedure.

LIST OF ABBREVIATIONS

ABBREVIATION	MEANING
API	Application Programming Interface
CSE (AI & ML)	Computer Science and Engineering (Artificial Intelligence and Machine Learning)
HOD	Head of Department
JWT	JSON Web Token
MERN	MongoDB, Express, React and Node.js
PDF	Portable Document Format
QR	Quick Response code
REST	Representational State Transfer
SMTP	Simple Mail Transfer Protocol
UI	User Interface

REFERENCES

[1] Digital Academic Document Workflow System, original project requirements and acceptance criteria, September 2026.

[2] Digital Academic Document Workflow System, project README and local startup guide, repository copy inspected 21 October 2026.

[3] Project source code: server/models/User.js, DocumentType.js, Request.js, GeneratedDocument.js and Notification.js.

[4] Project source code: server/controllers, server/routes, server/services/workflow.service.js and pdf.service.js.

[5] Project client source: client/src/pages, client/src/api and client/src/context.

[6] Easwari Engineering College, V Semester A Section timetable for 2026, supplied as context; used to confirm the CSE (AI & ML) department and Full Stack Development course context.

[7] React documentation, https://react.dev/.

[8] Vite documentation, https://vite.dev/.

[9] Express documentation, https://expressjs.com/.

[10] MongoDB documentation, https://www.mongodb.com/docs/.

[11] Mongoose documentation, https://mongoosejs.com/docs/.

[12] Node.js documentation, https://nodejs.org/docs/.

APPENDIX A — LOCAL STARTUP AND REVIEW

A.1 Start the Local Application

On the Windows machine containing the project, open PowerShell and run the project launcher from the repository folder:

```
cd "J:\My Drive\GOKUL LEARNING\Digital-Academic-Document-Workflow-System"
.\start-app.ps1
```

The launcher prepares a local runtime copy and opens the browser at http://127.0.0.1:5173. The server API listens on port 5000 in the current configuration. The runtime database and generated PDFs are stored under the user’s local application-data directory, rather than the Google Drive synchronized source folder.

A.2 Administrator Review Sequence

1. Sign in with the authorized administrator account. 2. Review Manage Users and confirm that DEMO accounts are clearly labelled. 3. Open Document Types to review each form and approval chain. 4. Open All Requests to inspect statuses and timelines. 5. Use a demo student account only in the local demonstration. 6. Verify a generated sample PDF through its QR code or verification route. Do not use demo accounts or the default local configuration as production credentials.

A.3 Repository Structure

The repository root contains the Windows launcher, README, client application and server application. Client pages are grouped by authentication, dashboard, administration, requests and verification. Server code is grouped into configuration, controllers, middleware, models, routes and services. The persistent runtime copy is separate from the source folder.

APPENDIX B — SAMPLE WORKFLOW RECORD

A typical Bonafide Certificate request uses the fields and stages defined by its document type. At submission, the request receives a unique document ID and an initial history event. Each approver action appends an event and advances the current stage. When the last required stage approves, the server generates a PDF, records its UID and QR data, marks the request complete and notifies the student. A verification request resolves that UID to its stored document record.

STEP	RECORD / ACTION	RESULT
1	Student submits a configured form	Request stored as submitted; first approver assigned.
2	Class Advisor reviews	Approval event recorded; next stage selected.
3	HOD reviews	Approval event recorded; final stage selected where configured.
4	Academic Office reviews	Final approval completes the request when this stage is configured.
5	PDF and QR record are created	Generated file path, unique document UID and verification URL are stored.
6	User scans or enters the UID	Verification page reports whether a matching valid record exists.

This sample is descriptive. The actual sequence is defined by the selected document type and may omit or add stages. A missing active approver should stop progression with an administrative message rather than silently bypass the stage.